

▼ Pooling

Muitas vezes, ao processar imagens, queremos gradualmente reduzir a resolução espacial das representações aprendidas (*feature maps*), agregando informações de forma que, quanto mais aprofundarmos na rede, maior o campo receptivo (*receptive field*).

Muitas vezes, a tarefa final é relacionada com alguma característica global da imagem como, por exemplo, na tarefa de classificação de cenas. Então, tipicamente, os neurônios da última camada devem conseguir captar informação da entrada como um todo. Ao agregar gradualmente as informações, produzindo *feature maps* de baixa resolução, alcançamos esse objetivo de aprender uma representação global, mantendo todas as vantagens das camadas convolucionais nas camadas intermediárias de processamento.

Além disso, ao detectar características de baixo nível, como bordas, muitas vezes queremos que as representações sejam um pouco invariantes à translação. Por exemplo, suponha uma imagem com uma definição nítida entre preto e branco. Suponha agora que deslocamos toda a imagem em um pixel para a direita. A saída para essa nova imagem pode ser muito diferente. A borda terá mudado em um pixel e, conseqüentemente, todas as ativações mudarão. Na realidade, os objetos quase nunca ocorrem exatamente no mesmo lugar. De fato, mesmo com um tripé e um objeto estacionário, vibração da câmera devido ao movimento do obturador pode mudar tudo por um pixel ou mais.

Nesta prática, veremos as camadas de pooling que tem dois propósitos básicas: (i) tornar a representação invariante à translação, e (ii) reduzir espacialmente as características aprendidas, aumentando o *receptive field*.

Max- e Mean-Pooling

Como camadas convolucionais, operadores de pooling consistem em uma janela (de tamanho fixo) deslizando sobre todas as regiões na entrada de acordo com seu *stride*, computando uma única saída para cada local visitado. No entanto, ao contrário das camadas convolucionais, a camada de pooling não tem parâmetros (ou seja, ela não aprende nada). Em vez disso, os operadores de pooling são determinísticos, normalmente calculando o valor máximo (*max*) ou médio (*mean*) dos elementos compreendido na sua janela. Essas operações são chamadas de *max-pooling* e *mean-pooling*, respectivamente.

Em ambos os casos, como na convolução, podemos pensar que o processo de pooling começa com sua janela no canto superior esquerdo da entrada e a desliza da esquerda para a direita e de cima para baixo. Em cada vizinhança delimitada pela janela, calcula-se o valor máximo ou médio dos pixels daquela região.



O array de saída da figura acima tem uma altura de 2 e uma largura de 2. Os quatro elementos são derivados do valor de máximo da vizinhança, ou seja:

$$\begin{aligned}\max(0, 1, 3, 4) &= 4, \\ \max(1, 2, 4, 5) &= 5, \\ \max(3, 4, 6, 7) &= 7, \\ \max(4, 5, 7, 8) &= 8.\end{aligned}$$

Vamos retornar ao exemplo de detecção de borda de objeto mencionado no início desta seção. Agora vamos usar a saída da camada convolucional como a entrada para um max-pooling 2×2 . Mesmo se a entrada para a camada convolucional se transladar um pixel para qualquer lado, a camada de pooling será capaz de gerar a mesma saída, já que avaliará a vizinhança para produzir a saída. Ou seja, usando uma camada de max-pooling de 2×2 , ainda podemos detectar o padrão reconhecido pela camada convolucional dado que este não se mova mais do que um pixel em altura e largura.

```
import time, os, sys, numpy as np
import torch
import torchvision
import torch.nn as nn
import torch.nn.functional as F

from torch import optim

import time, os, sys, numpy as np

# Test if GPU is available, if not, use cpu instead
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
n = torch.cuda.device_count()
devices_ids= list(range(n))
print(device)
```

cuda

```
def load_data_fashion_mnist(batch_size, resize=None, root=os.path.join(
    '~', '.pytorch', 'datasets', 'fashion-mnist')):
    """Download the Fashion-MNIST dataset and then load into memory."""
```

```

root = os.path.expanduser(root)
transformer = []
if resize:
    transformer += [torchvision.transforms.Resize(resize)]
transformer += [torchvision.transforms.ToTensor()]
transformer = torchvision.transforms.Compose(transformer)

mnist_train = torchvision.datasets.FashionMNIST(root=root, train=True, download=True, transform=transformer)
mnist_test = torchvision.datasets.FashionMNIST(root=root, train=False, download=True, transform=transformer)
num_workers = 0 if sys.platform.startswith('win32') else 4

train_iter = torch.utils.data.DataLoader(mnist_train,
                                          batch_size, shuffle=True,
                                          num_workers=num_workers)
test_iter = torch.utils.data.DataLoader(mnist_test,
                                         batch_size, shuffle=False,
                                         num_workers=num_workers)

return train_iter, test_iter

# funções básicas
def _get_batch(batch):
    """Return features and labels on ctx."""
    features, labels = batch
    if labels.type() != features.type():
        labels = labels.type(features.type())
    return (torch.nn.DataParallel(features, device_ids=devices_ids),
            torch.nn.DataParallel(labels, device_ids=devices_ids), features.shape[0])

# Função usada para calcular acurácia
def evaluate_accuracy(data_iter, net, loss):
    """Evaluate accuracy of a model on the given data set."""

    acc_sum, n, l = torch.Tensor([0]), 0, 0
    net.eval()
    with torch.no_grad():
        for X, y in data_iter:
            #y = y.astype('float32')
            X, y = X.to(device), y.to(device)
            y_hat = net(X)
            l += loss(y_hat, y).sum()
            acc_sum += (y_hat.argmax(axis=1) == y).sum().item()
            n += y.size()[0]

    return acc_sum.item() / n, l.item() / len(data_iter)

# Função usada no treinamento e validação da rede
def train_validate(net, train_iter, test_iter, batch_size, trainer, loss,
                  num_epochs):
    print('training on', device)
    for epoch in range(num_epochs):
        net.train()
        train_l_sum, train_acc_sum, n, start = 0.0, 0.0, 0, time.time()
        for X, y in train_iter:
            X, y = X.to(device), y.to(device)
            y_hat = net(X)
            trainer.zero_grad()
            l = loss(y_hat, y).sum()
            l.backward()
            trainer.step()
            train_l_sum += l.item()
            train_acc_sum += (y_hat.argmax(axis=1) == y).sum().item()
            n += y.size()[0]
        test_acc, test_loss = evaluate_accuracy(test_iter, net, loss)
        print('epoch %d, train loss %.4f, train acc %.3f, test loss %.4f, '
              'test acc %.3f, time %.1f sec'
              % (epoch + 1, train_l_sum / len(train_iter), train_acc_sum / n, test_loss,
                 test_acc, time.time() - start))

def weights_init(m):
    classname = m.__class__.__name__
    if classname.find('Conv') != -1:
        m.weight.data.normal_(0.0, 0.01)
    if classname.find('Linear') != -1:
        m.weight.data.normal_(0.0, 0.01)

```

Vamos agora, mostrar como funciona a camada de *pooling* na prática. Em frameworks modernos, camadas de *pooling* já vem implementadas e são fáceis de usar.

Abaixo, criamos uma matriz 2-D e a processamos usando um *max-pooling* de 2×2 .

```
X = torch.Tensor([[0, 1, 2], [3, 4, 5], [6, 7, 8]])
print(X)
X = X.reshape((1, 1) + X.shape)

pool = nn.MaxPool2d(kernel_size=2, stride=1)
y = pool(X)

print(y)
```

```
tensor([[[[0., 1., 2.],
          [3., 4., 5.],
          [6., 7., 8.]]]])
tensor([[[[4., 5.],
          [7., 8.]]]])
```

```
X = torch.Tensor([[0, 1, 2], [3, 4, 5], [6, 7, 8]])
print(X)
X = X.reshape((1, 1) + X.shape)

pool = nn.AvgPool2d(kernel_size=2, stride=1)
y = pool(X)

print(y)
```

```
tensor([[[[0., 1., 2.],
          [3., 4., 5.],
          [6., 7., 8.]]]])
tensor([[[[2., 3.],
          [5., 6.]]]])
```

Ao processar dados com múltiplos canais, a camada de pooling processa cada canal de entrada separadamente ao invés de processar todos os canais como em uma camada convolucional. Isso significa que o número de canais de saída para a camada de pooling é o mesmo que o número de canais de entrada. Abaixo, vamos concatenar X e X + 1 na dimensão do canal para construir uma entrada com 2 canais.

```
X = torch.cat((X, X + 1), dim=1)
X
```

```
tensor([[[[0., 1., 2.],
          [3., 4., 5.],
          [6., 7., 8.]],
         [[1., 2., 3.],
          [4., 5., 6.],
          [7., 8., 9.]]]])
```

```
pool2d = nn.MaxPool2d(kernel_size=2, stride=1)
pool2d(X)
```

```
tensor([[[[4., 5.],
          [7., 8.]],
         [[5., 6.],
          [8., 9.]]]])
```

✓ Padding and Stride

Como nas camadas convolucionais, as camadas de pooling também pode alterar as dimensões da saída. Da mesma forma que antes, podemos calcular a saída da camada baseada na sua configuração:

$$\lfloor (n_h - k_h + p_h) / s_h \rfloor \times \lfloor (n_w - k_w + p_w) / s_w \rfloor$$

E como antes, podemos configurar a operação para obter uma saída com dimensões desejadas usando *padding* e *stride*. Podemos demonstrar a influência de *padding* e *stride* em camadas de pooling através da camada de max-pooling *MaxPool2d* do framework Pytorch. Primeiro, construímos um dado de entrada com dimensões (1, 1, 4, 4), onde as duas primeiras dimensões são o tamanho do *batch* e canal.

```
X = torch.arange(16).reshape((1, 1, 4, 4))
X
```

```
tensor([[[[ 0, 1, 2, 3],
          [ 4, 5, 6, 7],
          [ 8, 9, 10, 11],
          [12, 13, 14, 15]]]])
```

Por padrão, o *stride* da *MaxPool2d* tem o mesmo tamanho da janela. Por exemplo, abaixo usamos uma janela de tamanho (3, 3). Como não especificamos explicitamente nenhum *stride*, obtemos um *stride* padrão de tamanho (3, 3).

```
pool2d = nn.MaxPool2d(kernel_size=3)
# Because there are no model parameters in the pooling layer, we do not need
# to call the parameter initialization function
pool2d(X.float()) # Only works with float

tensor([[[[10.]]]])
```

Logicamente, podemos especificar explicitamente o *padding* e o *stride* de uma camada de pooling.

```
pool2d = nn.MaxPool2d(kernel_size=3, padding=1, stride=2)
pool2d(X.float())

tensor([[[[ 5.,  7.],
           [13., 15.]]]]])
```

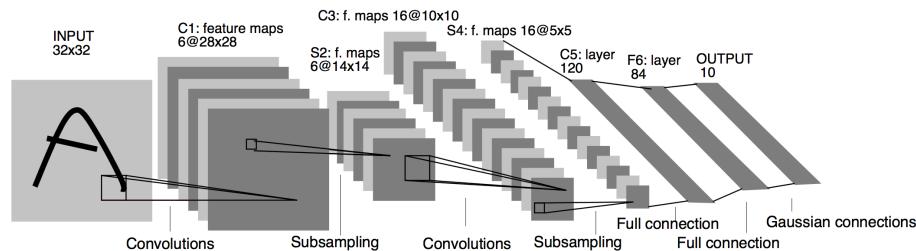
Podemos, também, especificar o tamanho de uma janela retangular arbitrária, do *padding* e do *stride* para altura e largura, respectivamente.

```
pool2d = nn.MaxPool2d(kernel_size=(2, 4), padding=(1, 2), stride=(2, 3))
pool2d(X.float())

tensor([[[[ 1.,  3.],
           [ 9., 11.],
           [13., 15.]]]]])
```

✓ Pytorch e o caso de estudo LeNet-5

Agora vamos implementar a [LeNet-5](#) completa usando Pytorch.



	Layer	Feature Map	Size	Kernel Size	Stride	Activation
	Input	Image	1	32x32	-	-
1	Convolution	6	28x28	5x5	1	tanh
2	Average Pooling	6	14x14	2x2	2	tanh
3	Convolution	16	10x10	5x5	1	tanh
4	Average Pooling	16	5x5	2x2	2	tanh
5	Convolution	120	1x1	5x5	1	tanh
6	FC	-	84	-	-	tanh
Output	FC	-	10	-	-	softmax

```
# parâmetros: número de epochs, learning rate (ou taxa de aprendizado),
# tamanho do batch, e lambda do weight decay
num_epochs, lr, batch_size, wd_lambda = 10, 0.1, 128, 0.00001

# rede baseada na LeNet-5
net = nn.Sequential(nn.Conv2d(in_channels=1,out_channels=6, kernel_size=5, stride=1, padding=0), # entrada: (b, 1, 32, 32)
                    nn.Tanh(),
                    nn.AvgPool2d(kernel_size=2, stride=2, padding=0), # entrada: (b, 6, 28, 28) e saída: (b, 6, 14, 14)
                    nn.Conv2d(in_channels=6,out_channels=16, kernel_size=5, stride=1, padding=0), # entrada: (b, 6, 14, 14) e saída: (b, 16, 10, 10)
                    nn.Tanh(),
                    nn.AvgPool2d(kernel_size=2, stride=2, padding=0), # entrada: (b, 16, 10, 10) e saída: (b, 16, 5, 5)
                    nn.Conv2d(in_channels=16,out_channels=120, kernel_size=5, stride=1, padding=0), # entrada: (b, 16, 5, 5) e saída: (b, 120, 1, 1)
                    nn.Tanh(),
                    nn.Linear(120, 84), # entrada: (b, 120) e saída: (b, 84)
                    nn.Tanh(),
                    nn.Linear(84, 10), # entrada: (b, 84) e saída: (b, 10)
                    nn.Softmax())
```

```
nn.Flatten(), # lineariza formando um vetor # entrada: (0, 120, 1, 1) e saída: (0, 120*1)
nn.Linear(120, 84), # entrada: (b, 120) e saída: (b, 84)
nn.Tanh(),
nn.Linear(84,10))

# entrada: (b, 84) e saída: (b, 10)

# Sending model to device
net.to(device)

# função de custo (ou loss)
loss = nn.CrossEntropyLoss()

# carregamento do dado: mnist
train_iter, test_iter = load_data_fashion_mnist(batch_size, resize=32)

# trainer do gluon
trainer = optim.SGD(net.parameters(), lr=lr, weight_decay=wd_lambda, momentum=0.9)

# treinamento e validação via Pytorch
train_validate(net, train_iter, test_iter, batch_size, trainer, loss,
               num_epochs)
```

```
training on cuda
epoch 1, train loss 0.6242, train acc 0.767, test loss 0.5004, test acc 0.815, time 17.4 sec
epoch 2, train loss 0.3961, train acc 0.854, test loss 0.4017, test acc 0.850, time 11.9 sec
epoch 3, train loss 0.3530, train acc 0.869, test loss 0.3696, test acc 0.869, time 12.2 sec
epoch 4, train loss 0.3214, train acc 0.880, test loss 0.3487, test acc 0.870, time 11.8 sec
epoch 5, train loss 0.3024, train acc 0.888, test loss 0.3436, test acc 0.879, time 11.4 sec
epoch 6, train loss 0.2852, train acc 0.894, test loss 0.3290, test acc 0.879, time 11.5 sec
epoch 7, train loss 0.2685, train acc 0.900, test loss 0.3362, test acc 0.883, time 12.0 sec
epoch 8, train loss 0.2631, train acc 0.902, test loss 0.3193, test acc 0.888, time 12.2 sec
epoch 9, train loss 0.2483, train acc 0.907, test loss 0.3248, test acc 0.886, time 12.4 sec
epoch 10, train loss 0.2424, train acc 0.910, test loss 0.3256, test acc 0.886, time 12.8 sec
```